

Ex No. 10 Ansible Installation and Create a sample Ansible playbook

1. Install Ansible on your control node using the system package manager.

```
$ sudo apt update
```

```
$ sudo apt install software-properties-common
```

```
$ sudo add-apt-repository --yes --update ppa:ansible/ansible
```

```
$ sudo apt install ansible
```

Error/Warning: If user not in the sudoers list, run the following commands

- su
- sudo usermod -aG sudo "username"
- su "username"

2. Ensure Ansible is installed successfully

```
ansible --version
```

Error: UTF-8 encoding expected for ansible, run the following commands

- sudo nano /etc/default/locale
- Update the encoding to UTF-8

```
LANG="en_US.UTF-8"
```

```
LC_CTYPE="en_US.UTF-8"
```

(Or)

- Run the below command in terminal

```
sudo update-locale LANG=en_US.UTF-8 LC_CTYPE=en_US.UTF-8
```

- Restart

3. Creating SSH Key - Creates a public/private SSH key pair for secure password

less access.

```
ssh-keygen -t rsa -C "devops default"
```

```
ls -la .ssh
```

```
cat .ssh/id_rsa.pub
```

```
cat .ssh/id_rsa
```

4. Copying SSH Key to server - Enables password less SSH login by copying the key to managed nodes.

```
ssh-copy-id -i ~/.ssh/id_rsa.pub "ipaddress"
```

5. Testing the new Key - Verifies successful login to remote servers without a password.

```
ssh -i ~/.ssh/"keyname" "ipaddress"
```

Error: port 22, Connection Refused, run the following commands

```
sudo apt install openssh-server
```

```
sudo systemctl restart sshd
```

6. Connecting to Server

```
ssh "Ip address"
```

7. Create Inventory File - Lists all managed servers in a structured format for Ansible to use.

```
sudo nano inventory
```

```
[servers]
```

```
192.168.1.219
```

```
192.168.1.218
```

8. Ansible Command to connect with repository provided in the inventory

```
ansible all --key-file ~/.ssh/id_rsa -i inventory -m ping
```

Error: If permission Denied, run – “chmod 600 ~/.ssh

Inventory Testing - ansible all -i inventory -m ping

9. Creating playbook named playbook.yml

```
sudo nano playbook.yml
```

```
playbook.yml
```

```
---
```

```
- name: Echo
```

```
hosts: servers
```

```
connection: local
```

```
tasks:
```

```
- name: print ping message
```

```
debug:
```

```
msg: Hello!
```

10. Executing playbook

```
ansible-playbook -i inventory playbook.yml
```

```
netstat -tuln
```

azure pipeline.yml

```
trigger:
```

```
- main
```

```
pool: test
```

```
- task: Maven@3
```

```
inputs:
```

```
mavenPomFile: 'pom.xml'
```

```
mavenOptions: '-Xmx3072m'
```

```
javaHomeOption: 'Path'
```

```
jdkDirectory: 'C:\Program Files\jdk-21.0.7'
```

```
jdkArchitectureOption: 'x64'
```

```
publishJUnitResults: true
```

```
testResultsFiles: '**/surefire-reports/TEST-*.xml'
```

```
goals: 'compile'
```

Basic Git Commands

```
git init
```

```
git config --global --list
```

```
git config --global --user.name
```

```
git add "file name"
```

```
git add.
```

```
git commit -m "msg"
```

git status

git log

git branch -m main

git remote add origin "URL"

git push -m origin "branch name"

git checkout -b "new branch name"

git switch -c "new branch name"

git checkout/switch "branch name"

git merge "source branch name"

git pull origin "branch name"

git tag "tag name"

git tag -a "tag name" -m "message"

git push origin tag name v

git.txt

git --version

git config --global --list

git config --global user.name ""

git config --global user.email ""

git init

git add

git commit -m "initial gradle

git branch -M main

git remote add origin <https://github.com/danicaasilva/gradle.git>

git push-u origin main

git checkout -b feature-branch

git add

git commit -m "feature branch change

git push origin feature-branch

git checkout main

git add commit -m "main branch conflict change"

git push origin main

git merge feature-branch

git add build.gradle

git commit -m "Resolved"

git push origin main

src/main/java/com/example/demo/

👉 Create 3 packages:

controller

service

model

✅ 4. Code Implementation

◆ 1. DemoApplication.java (Already exists)

📌 Location:

src/main/java/com/example/demo/DemoApplication.java

```
package com.example.demo;
```

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
public class DemoApplication {
```

```
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

◆ 2. Model (User.java)

📌 Create:

model/User.java

```
package com.example.demo.model;
```

```
public class User {
```

```
    private String username;
    private String password;
```

```
    // Getter & Setter
```

```
    public String getUsername() {
        return username;
    }
}
```

```
public void setUsername(String username) {
    this.username = username;
}

public String getPassword() {
    return password;
}

public void setPassword(String password) {
    this.password = password;
}
}
```

◆ 3. Service (LoginService.java)

📌 Create:

service/LoginService.java

```
package com.example.demo.service;
```

```
import org.springframework.stereotype.Service;
```

```
@Service
```

```
public class LoginService {
```

```
    public boolean validate(String username, String password) {
        // Simple hardcoded login
        return username.equals("admin") && password.equals("1234");
    }
}
```

◆ 4. Controller (LoginController.java)

📌 Create:

controller/LoginController.java

```
package com.example.demo.controller;
```

```
import com.example.demo.model.User;
```

```
import com.example.demo.service.LoginService;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.stereotype.Controller;
```

```
import org.springframework.ui.Model;
```

```
import org.springframework.web.bind.annotation.*;
```

```
@Controller
```

```

public class LoginController {

    @Autowired
    private LoginService loginService;

    // Show login page
    @GetMapping("/")
    public String showLoginPage(Model model) {
        model.addAttribute("user", new User());
        return "login";
    }

    // Handle login
    @PostMapping("/login")
    public String login(@ModelAttribute User user, Model model) {

        boolean isValid = loginService.validate(user.getUsername(), user.getPassword());

        if (isValid) {
            model.addAttribute("message", "Login Successful!");
            return "success";
        } else {
            model.addAttribute("message", "Invalid Username or Password");
            return "login";
        }
    }
}

```

5. HTML Files (View)

 Location:

src/main/resources/templates/

login.html

```

<!DOCTYPE html>
<html>
<head>
    <title>Login Page</title>
</head>
<body>

<h2>Login</h2>

<form action="/login" method="post">
    Username: <input type="text" name="username" /><br><br>

```



```
    Password: <input type="password" name="password" /><br><br>
    <button type="submit">Login</button>
</form>

<p style="color:red;">${message}</p>

</body>
</html>
```

◆ success.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Success</title>
</head>
<body>

<h2>${message}</h2>

</body>
</html>
```

✅ 6. application.properties

📍 Location:

src/main/resources/application.properties

Add:

```
spring.thymeleaf.cache=false
server.port=8080
```

✅ 7. Add Dependency (IMPORTANT)

Open pom.xml and make sure you have:

```
<dependencies>

    <!-- Spring Boot Web -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- Thymeleaf (for HTML) -->
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

```
</dependencies>
```

✅ 8. Run the Project

In VS Code terminal:

```
mvn spring-boot:run
```

✅ 9. Open Browser

Go to:

```
http://localhost:8080
```

Gradle

```
gradle -v
```

👉 If it shows version → SUCCESS ✅

📁 STEP 3: Create Your First Gradle Project

Create a folder:

```
mkdir myapp
cd myapp
```

Initialize Gradle project:

```
gradle init
```

You'll be asked:

- Type → application
- Language → java

- Build script → Groovy
- Package → com.example
- Project name → myapp

👉 Done! Gradle creates everything automatically.

📁 Project Structure (Important)

```
myapp/
├── build.gradle
├── settings.gradle
├── gradlew
├── src/
│   ├── main/java/com/example/App.java
│   └── test/
```

📖 STEP 4: Understand Key Files

◆ build.gradle

Main configuration file

Example:

```
plugins {
    id 'application'
}

repositories {
    mavenCentral()
}

dependencies {
    testImplementation 'junit:junit:4.13.2'
}

application {
    mainClass = 'com.example.App'
}
```

▶ STEP 5: Build the Project

Run:

gradle build

👉 This compiles your code

▶ STEP 6: Run the Project

Run:

gradle run

👉 Output:

Hello Gradle!

⚡ IMPORTANT COMMANDS

Command	Use
---------	-----

gradle build	Compile project
--------------	-----------------

gradle run	Run app
------------	---------

gradle clean	Delete build files
--------------	--------------------

gradle tasks	Show all commands
--------------	-------------------

💡 STEP 7: Using Gradle in VS Code

Install extensions:

- Java Extension Pack
 - Gradle for Java
-

Open your folder:

File → Open Folder → myapp

👉 You can now run from VS Code

1. Run gradle init
 2. Use gradle build
 3. Use gradle run
-

🚀 STEP 1: Replace Your App.java

👉 Go to:

src/main/java/com/example/App.java

Replace everything with this:

```
package com.example;

import java.util.Scanner;

public class App {
    public static void main(String[] args) {

        // Predefined username and password
        String correctUsername = "admin";
        String correctPassword = "1234";

        Scanner sc = new Scanner(System.in);

        System.out.println("=== LOGIN SYSTEM ===");

        // Input from user
        System.out.print("Enter Username: ");
        String username = sc.nextLine();

        System.out.print("Enter Password: ");
        String password = sc.nextLine();

        // Check login
        if (username.equals(correctUsername) && password.equals(correctPassword)) {
            System.out.println("✅ Login Successful!");
        } else {
            System.out.println("❌ Invalid Username or Password");
        }

        sc.close();
    }
}
```

STEP 2: Run the Project

Open terminal in project folder:

```
gradle run
```

OUTPUT

```
=== LOGIN SYSTEM ===
Enter Username: admin
```

Enter Password: 1234

 Login Successful!

 **WRONG INPUT**

Enter Username: user

Enter Password: 0000

 Invalid Username or Password